

FUNCTIONAL REQUIREMENTS DOCUMENT (FRD)

Phase 1: Multi-Tenant Base Architecture, Core Pipelines, Customization Engines, and Foundation Matrix

System Blueprint Specification Sheet

Designed for Software Engineering, Database Administration, and UI/UX Development Teams
Comprehensive Execution Guide: Translating Spreadsheet Operations into Type-Safe Relational Software Engines

DOCUMENT CONTROL METADATA:

Project Framework: Custom AI-Native Revenue OS (CRM)	Development Phase: Phase 1 MVP Core Baseline Implementation
Target Timeline: First 60 Calendar Days Execution	Database Engine: Supabase PostgreSQL (Fully Federated & Custom Field Elastic)
API Layer Standard: NestJS Enterprise Micro-modules with Prisma ORM	Frontend Interface: Next.js 15 (App Router, TypeScript, shadcn/ui, TanStack Table)

I. PRODUCT OVERVIEW & CORE VISION

1.1 Objective Statement

The primary goal of this system is to fully transition manual sales tracking workflows out of fragmented Excel spreadsheets, paper notebooks, and isolated calendars into a unified application. Phase 1 provides the structural foundation required to run a high-velocity, multi-tenant sales pipeline. It establishes clean relational data boundaries, eliminates manual record duplication, enforces time-bound follow-up accountability via system alerts, and creates an elastic schema foundation capable of supporting advanced background AI pipelines in subsequent phases.

1.2 User Personas & Authorization Matrix

The platform natively supports strict multi-tenancy. This guarantees that multiple independent organizations can register and utilize the software safely, while Row-Level Security (RLS) ensures that Company A can never view, alter, or access the data streams of Company B. Within each company organization, users are bound to specific functional roles defined below:

- Super Admin:** Platform owners with universal infrastructure visibility. They manage master subscription configurations, system-wide parameters, and cross-tenant maintenance blocks.
- Organization Admin (Company Owner):** Possesses complete data control inside their specific tenant. They manage team permissions, customize pipeline stages, define custom database attributes, and configure third-party application hooks.
- Sales Manager:** Monitored dashboard access to oversee pipeline movement. They evaluate team performance metrics, handle deal assignments, override sales stages, and generate executive pipeline reports.

- **Sales Representative:** The primary front-line execution role. They view and manage self-assigned or routed leads, execute pipeline stages, create client notes, log interaction activities, and complete task checklists. They cannot view records owned by other representatives unless granted permission.
- **Viewer:** Read-only operational visibility. Used by auxiliary staff or legal stakeholders to verify information states without the ability to modify fields or execute workflows.

II. CORE TECHNOLOGY STACK ARCHITECTURE

To ensure high-speed performance, clean data modeling, and seamless support for future AI modules, the system uses a fully typed and unified technology stack:

ARCHITECTURAL LAYER	SELECTED TOOL / TECHNOLOGY	STRATEGIC TECHNICAL RATIONALE
Frontend Engine	Next.js 15 (App Router) & TypeScript	Provides fast Server-Side Rendering (SSR) for landing pages and Client-Side rendering for dynamic dashboard widgets. Full type-safety across views.
UI Base System	shadcn/ui & Tailwind CSS	Ensures a clean, professional, un-styled component baseline allowing full style flexibility without bloat.
Data Table Engine	TanStack Table (v8)	Handles high-performance data rows natively, enabling client-side sorting, column ordering, and multi-selection grouping out-of-the-box.
Drag & Drop Logic	@dnd-kit/core	Provides smooth, high-frame-rate drag-and-drop mechanics for shifting deal cards across Kanban columns.
Backend Framework	NestJS (Enterprise Architecture Pattern)	Enforces a strict dependency injection pattern, splitting modules cleanly (e.g., LeadModule, CustomFieldModule) for high maintainability.
Database & Infrastructure	Supabase PostgreSQL Engine	Combines relational data integrity, native user authentication APIs, cloud storage, and robust Row-Level Security (RLS) under a single platform.
ORM Layer	Prisma ORM	Generates autocompleted, type-safe data queries and migrations directly from the core database schema.
Caching & Async Queues	Redis Cloud & BullMQ	Manages heavy asynchronous background execution lines, such as system alert notifications and status calculation caches.
Communication Nodes	Resend API Engine	A reliable, developer-friendly transactional email node used for security confirmations, alerts, and system notifications.
Charts & Visualization	Recharts Library	Renders fluid, SVG-rendered reporting assets and pipelines that adapt perfectly across fluid dashboard windows.

III. PHASE 1 DATABASE ARCHITECTURE & PHYSICAL SCHEMAS

The following design maps out the physical PostgreSQL relational architecture. It enforces strict data constraints, cascade operations, and indexing strategies to keep page response speeds under 500ms even across millions of rows.

3.1 Primary Schema Definitions (Prisma Schema Model Format)

This script represents the structural truth of the underlying database layers used during the Prisma migration process:

```
// 1. Tenant Control Object
model Organization {
  id          String    @id @default(uuid())
  name        String
  slug        String    @unique
  industry    String?
  website     String?
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

  users       User[]
  leads       Lead[]
  deals       Deal[]
  tasks       Task[]
  notes       Note[]
  activities  Activity[]
  customFields CustomFieldDefinition[]

  @@map("organizations")
}

// 2. Identity & Access Layer
enum UserRole {
  SUPER_ADMIN
  ORG_ADMIN
  SALES_MANAGER
  SALES_REP
  VIEWER
}

model User {
  id          String    @id @default(uuid())
  organizationId String    @map("organization_id")
  email       String    @unique
  fullName    String    @map("full_name")
  role        UserRole   @default(SALES_REP)
  phone       String?
  status      String     @default("ACTIVE") // ACTIVE, SUSPENDED
  createdAt   DateTime   @default(now()) @map("created_at")

  organization Organization @relation(fields: [organizationId], references: [id], onDelete: Cascade)
  ownedLeads   Lead[]       @relation("LeadOwner")
  ownedDeals   Deal[]       @relation("DealOwner")

  @@index([organizationId])
  @@map("users")
}
```

```
// 3. Core Lead Ingestion Record
enum LeadStatus {
    NEW
    CONTACTED
    INTERESTED
    QUALIFIED
    REJECTED
}

model Lead {
    id                String          @id @default(uuid())
    organizationId    String          @map("organization_id")
    firstName         String          @map("first_name")
    lastName          String          @map("last_name")
    email             String?
    phone             String?
    company           String?
    source            String          @default("DIRECT") // OUTBOUND, ADS, WEBFORM
    status            LeadStatus      @default(NEW)
    ownerId           String?         @map("owner_id")
    createdAt         DateTime        @default(now()) @map("created_at")
    updatedAt         DateTime        @updatedAt @map("updated_at")

    organization      Organization @relation(fields: [organizationId], references: [id], onDelete:
Cascade)
    owner             User?         @relation("LeadOwner", fields: [ownerId], references: [id],
onDelete: SetNull)
    prospectRecord    Prospect?
    deals             Deal[]
    customValues      CustomFieldValue[]

    @@index([organizationId])
    @@index([ownerId])
    @@index([status])
    @@map("leads")
}

// 4. Prospect Qualification Framework (BANT Standard Elements)
model Prospect {
    id                String          @id @default(uuid())
    leadId            String          @unique @map("lead_id")
    budget            Decimal?        @db.Decimal(15, 2)
    authority          Boolean         @default(false)
    need              String?         @db.Text
    timeline          String?         // Expected purchase window description
    qualifiedBy       String          @map("qualified_by")
    qualifiedAt       DateTime        @default(now()) @map("qualified_at")

    lead              Lead            @relation(fields: [leadId], references: [id], onDelete: Cascade)
    deals             Deal[]

    @@map("prospects")
}
```

```
// 5. Sales Deal Module
enum DealStage {
  NEW
  PROPOSAL
  NEGOTIATION
  CONTRACT
  WON
  LOST
}

model Deal {
  id                String          @id @default(uuid())
  organizationId    String          @map("organization_id")
  leadId            String          @map("lead_id")
  prospectId        String?         @map("prospect_id")
  title             String
  value             Decimal          @db.Decimal(15, 2)
  stage             DealStage       @default(NEW)
  probability        Int             @default(10) // Percentage integer (0 - 100)
  ownerId           String?         @map("owner_id")
  expectedCloseDate DateTime?       @map("expected_close_date")
  createdAt         DateTime        @default(now()) @map("created_at")
  updatedAt         DateTime        @updatedAt @map("updated_at")

  organization      Organization @relation(fields: [organizationId], references: [id],
onDelete: Cascade)
  lead              Lead         @relation(fields: [leadId], references: [id], onDelete:
Cascade)
  prospect           Prospect?   @relation(fields: [prospectId], references: [id], onDelete:
SetNull)
  owner              User?       @relation("DealOwner", fields: [ownerId], references: [id],
onDelete: SetNull)
  contracts          Contract[]

  @@index([organizationId])
  @@index([stage])
  @@map("deals")
}

// 6. Contract Execution Module
enum ContractStatus {
  DRAFT
  SENT
  SIGNED
  EXPIRED
}

model Contract {
  id                String          @id @default(uuid())
  dealId            String          @map("deal_id")
  contractNumber    String          @unique @map("contract_number")
  status            ContractStatus @default(DRAFT)
  fileUrl           String          @map("file_url") // Location string in Supabase Storage Bucket
  signedAt          DateTime?       @map("signed_at")
  expiresAt         DateTime?       @map("expires_at")
}
```

```
    createdAt      DateTime      @default(now()) @map("created_at")

    deal           Deal           @relation(fields: [dealId], references: [id], onDelete:
Cascade)
    customers      Customer[]

    @@map("contracts")
}

// 7. Post-Contract Active Customer Directory
model Customer {
  id              String         @id @default(uuid())
  organizationId  String         @map("organization_id")
  contractId      String         @map("contract_id")
  customerSince   DateTime       @default(now()) @map("customer_since")
  status          String         @default("ACTIVE") // ACTIVE, CHURNED

  contract        Contract @relation(fields: [contractId], references: [id], onDelete: Cascade)

  @@map("customers")
}
```

```
// 8. Tasks, Communication Feeds, and System Notes Schema
enum TaskPriority {
    LOW
    MEDIUM
    HIGH
}

model Task {
    id                String          @id @default(uuid())
    organizationId    String          @map("organization_id")
    title             String
    description        String?        @db.Text
    dueDate           DateTime        @map("due_date")
    priority           TaskPriority    @default(MEDIUM)
    assignedTo        String          @map("assigned_to")
    relatedType        String          @map("related_type") // LEADS, DEALS, etc.
    relatedId         String          @map("related_id")
    isCompleted        Boolean        @default(false) @map("is_completed")
    createdAt         DateTime        @default(now()) @map("created_at")

    organization      Organization @relation(fields: [organizationId], references: [id], onDelete:
Cascade)

    @@index([organizationId, assignedTo])
    @@map("tasks")
}

model Note {
    id                String          @id @default(uuid())
    organizationId    String          @map("organization_id")
    content           String          @db.Text
    createdBy         String          @map("created_by")
    relatedType        String          @map("related_type") // LEADS, DEALS
    relatedId         String          @map("related_id")
    createdAt         DateTime        @default(now()) @map("created_at")

    organization      Organization @relation(fields: [organizationId], references: [id], onDelete:
Cascade)

    @@index([relatedType, relatedId])
    @@map("notes")
}

model Activity {
    id                String          @id @default(uuid())
    organizationId    String          @map("organization_id")
    type              String          @map("type") // CALL, EMAIL, STATUS_CHANGE, TASK_DONE
    userId            String          @map("user_id")
    relatedType        String          @map("related_type")
    relatedId         String          @map("related_id")
    description        String          @db.Text
    createdAt         DateTime        @default(now()) @map("created_at")

    organization      Organization @relation(fields: [organizationId], references: [id], onDelete:
Cascade)
}
```

```

    @@index([organizationId])
    @@map("activities")
  }

```

3.2 Custom Field Extension Engine Schemas

To support dynamic field additions without breaking relational boundaries or forcing physical column alterations, the database uses an entity-attribute-value schema pattern:

```

enum FieldType {
  TEXT
  NUMBER
  DATE
  BOOLEAN
}

model CustomFieldDefinition {
  id          String          @id @default(uuid())
  organizationId String       @map("organization_id")
  fieldName   String         @map("field_name") // Internal key slug (e.g. "gst_number")
  fieldLabel  String         @map("field_label") // Visual UI label (e.g. "GST Number")
  fieldType   FieldType       @default(TEXT)
  relatedModule String       @map("related_module") // LEADS, DEALS
  createdAt   DateTime       @default(now()) @map("created_at")

  organization Organization @relation(fields: [organizationId], references: [id],
onDelete: Cascade)
  values          CustomFieldValue[]

  @@unique([organizationId, fieldName, relatedModule])
  @@map("custom_field_definitions")
}

model CustomFieldValue {
  id          String          @id @default(uuid())
  definitionId String         @map("definition_id")
  leadId      String         @map("lead_id")
  value       String         @db.Text // All primitive entries stored as string formats

  definition CustomFieldDefinition @relation(fields: [definitionId], references: [id],
onDelete: Cascade)
  lead       Lead                  @relation(fields: [leadId], references: [id], onDelete:
Cascade)

  @@unique([definitionId, leadId])
  @@map("custom_field_values")
}

```

IV. FRONTEND APPLICATION LAYOUT & FEATURE ROUTES

This layout outlines the explicit screen layouts, interactive views, page mechanics, and dashboard widgets required across the user path. The authentication boundary partitions public layers from the main logged-in dashboard system.

4.1 Public Marketing & Authentication Layer

4.1.1 Public Landing Page (^)

- **Hero Display:** Professional introductory section stating the platform value proposition. Prominent conversion buttons: `[Sign Up Free Trial]` and `[Book Live Architecture Demo]`.
- **Interactive Features Matrix:** Modern layout highlighting core Phase 1 modules (Leads, Pipelines, Analytics, Customization Engine) and preview tags for subsequent AI modules.
- **Tiered Pricing Grid:** Clearly displaying Starter, Growth, and Enterprise tiers. Clicking any tier passes selection parameters straight to the onboarding layout.
- **Onboarding Flow Form:** Captures user and company details to instantiate the tenant workspace profile.

4.1.2 Auth Access Channels (^/auth/login` & ^/auth/register`)

- **Sign-In Module (^/auth/login`):** Standard, centered authentication box containing:
 - Email/Password standard form inputs with integrated frontend string structure checks.
 - One-Click OAuth buttons for Google and Microsoft Account single-sign-on credentials.
 - Fallback One-Time Password (OTP) validation framework sent directly via backend communication nodes.
- **Sign-Up Registration Module (^/auth/register`):** A multi-step form built to capture organizational parameters:
 - Step 1: Account Owner Identity Data (Full Name, Personal Work Email, Direct Contact Phone Number, Login Password).
 - Step 2: Corporate Entity Profile (Company Legal Name, Targeted Industry Vertical, Company Employee Count, Base Currency Choice).
 - Step 3: Background Initiation Block. Submitting the form handles backend setup sequences securely: provisions an isolated Organization row, generates the Admin user profile, applies standard default deal stages, and redirects the session to the main login portal.

4.2 Logged-In Application Layout (Global Framework Shell)

Once a session is verified via Supabase Auth tokens, the layout wraps the inner dashboard views inside a standard administrative template. This framework features a constant navigation sidebar and a global tracking header:



CONSTANT APPLICATION HEADER ELEMENTS

- **Global Unified Search Box:** A centralized keyboard-accessible command box. Typing strings matches parameters against full system names, phones, emails, and deal identifiers using sub-second lookup hooks.
- **Contextual Notification Panel:** A toggle window tracking real-time status updates, overdue task flags, or system reminders.
- **User Settings Workspace:** Pop-up dropdown containing personal account settings, profile adjustments, role reviews, and the logout terminal button.

4.2.1 Left Navigation Sidebar Architecture

The global left sidebar houses primary workspace navigation routes. Links change based on user access levels:

```
├─ Navigation Links Shell
│   ├─ Dashboard Core Terminal (`/dashboard`)
│   ├─ Leads Intake Management Engine (`/dashboard/leads`)
│   ├─ Qualified Contact Profiles Engine (`/dashboard/contacts`)
│   ├─ Prospect Qualification BANT Workspace (`/dashboard/prospects`)
│   ├─ Visual Deal Pipeline Boards (`/dashboard/deals`)
│   ├─ Contract Storage & Tracking Panel (`/dashboard/contracts`)
│   ├─ Active Customers Retainer Panel (`/dashboard/customers`)
│   ├─ Task Checklist & Schedules Tracker (`/dashboard/tasks`)
│   ├─ Chronological Global Activity Logs (`/dashboard/activities`)
│   ├─ Performance Reports & Analytics Boards (`/dashboard/analytics`)
│   ├─ Internal Team Onboarding & Roles Area (`/dashboard/team`)
│   └─ System Customization & Configuration Workspace (`/dashboard/settings` – Admin Only)
```

4.3 Detailed Feature Pages & Layout Specifications

4.3.1 Central Operations Control Terminal (`/dashboard`)

The layout acts as an aggregated data console displaying widgets tailored to the logged-in user's role. It displays performance metrics across five primary categories:

- Financial Pipeline Performance Widgets:** Displays Total Confirmed Closed Revenue, Month-to-Date Accrued Inbound Pipeline, and Forecasted Weighted Revenue totals based on stage percentages.
- Lead Velocity Widgets:** Monitors total count flags for New Leads Captured, Today's Intake Volume, and Qualified Conversion Rates.
- Conversion Funnel Drop-off Trackers:** Displays visual conversion progression rates tracking opportunities across primary business milestones (Lead → Prospect → Opportunity → Contract → Signed Customer).
- Daily Activity Counters:** Real-time counts showing total call logs made, outreach emails sent, and team notes generated during the active shift.
- Team Leaderboard Ranking:** Displays ranking lists tracking won revenue volumes across active sales reps (visible to Managers and Admins).

4.3.2 Leads Intake Management Module (`/dashboard/leads`)

This workspace serves as the primary system entry point. To support high-speed processing, the workspace supports **Multi-View UI States** toggled instantly using frontend controls:

- **Table View Structure:** Powered by TanStack Table. Renders columns for: `Name`, `Phone/Email`, `Marketing Source`, `Status Badge`, `Assigned Rep`, and `Creation Timestamp`. Includes inline multi-select checkboxes for bulk task assignments or owner updates.
- **Visual Kanban Board View:** Segregates lead entries into clean status-matched columns: `[NEW]`, `[CONTACTED]`, `[INTERESTED]`, and `[QUALIFIED]`. Moving a card across columns updates records instantly via drag-and-drop webhooks.
- **Card/Grid View layouts:** Displays a modular layout presenting compact info tiles, designed to optimize data discovery across mobile screens.

Lead Detail Profile Workspace (`/dashboard/leads/[id]`): A dual-column contextual profile designed to surface full account history without switching pages:

- **Left Metadata Panel:** Contains baseline identity inputs, edit buttons, core lead properties, and dynamically rendered **Custom Field Inputs** configured by the company admin.
- **Right Activity Timeline Workspace:** Contains interactive tabs:
 - **Overview:** A summary view aggregating recent notes, recent tasks, and contact info highlights.
 - **Timeline Feed:** An unalterable log displaying interactions across email, phone data, and status logs sorted by timestamp.
 - **Task Manager:** Task tracking module allowing reps to create reminders, set deadlines, and check off completed items.
 - **Document Storage:** Document uploading node connecting directly to Supabase Storage Buckets to house client materials.

4.3.3 Qualified Contact Database Workspace (`/dashboard/contacts`)

- **Unified Database List:** Displays individual customer and stakeholder listings. Supports advanced server-side data sorting, company grouping lookups, and tag-filtering capabilities.
- **Contact Detail Workspace (`/dashboard/contacts/[id]`):** Profiles corporate buyers and lists all historical interaction touches, cross-linked open deals, past signatures, and account activities.

4.3.4 Prospect Qualification & BANT Interface (`/dashboard/prospects`)

This module acts as the qualification gate before a lead is authorized to advance to an open deal. It uses the enterprise **BANT Qualification Framework** to log operational requirements:

- **Prospect Validation Form:** Mandatory fields tracking:
 - **Budget Check:** Numerical currency input fields verifying available client funding.
 - **Authority Verification:** Binary check verifying communication with decision-making stakeholders.
 - **Need Identification:** Free-form text fields detailing explicit pain points requiring resolution.
 - **Timeline Schedule:** Timeframe indicator field defining expected implementation schedules.

4.3.5 Visual Deal Pipeline Board (`/dashboard/deals`)

- **Core Pipeline Board:** A drag-and-drop board displaying deals across primary sales milestones: `[NEW]` , `[PROPOSAL]` , `[NEGOTIATION]` , `[CONTRACT]` , `[WON]` , and `[LOST]` . Deal card headers display Title, Company, Total Value, and Owner. Columns dynamically update deal value summaries.
- **Forecast Pipeline View:** Adjusts visible deal values based on their stage assignment probability weights, enabling sales leadership to predict near-term revenue pipelines.

4.3.6 Contract Storage & Tracking Panel (`/dashboard/contracts`)

- **Contract Log Dashboard:** Tracks document states across production stages: `[DRAFT]` , `[SENT]` , `[SIGNED]` , and `[EXPIRED]` .
- **Integrated Viewer Panel (`/dashboard/contracts/[id]`):** Displays contracts via web-safe PDF wrappers alongside detailed e-signature logs, version tracking controls, and explicit execution timestamps.

4.3.7 Active Customers Retainer Panel (`/dashboard/customers`)

- **Customer Directory:** Displays all accounts with fully executed contracts. Connects to project delivery files, calculates total client value metrics, and logs customer success touches.

4.3.8 Task Checklist & Schedules Tracker (`/dashboard/tasks`)

- **Task Grid interface:** Consolidates multi-module tasks across list, visual board, or interactive calendar layouts. Reps categorize actions by type (`CALL` , `MEETING` , `EMAIL` , `FOLLOW-UP`) and filter schedules by deadline or priority flags.

4.3.9 Performance Reports & Analytics Boards (`/dashboard/analytics`)

Provides deep analytical insight using interactive chart blocks:

- **Lead Funnel Discrepancy Charts:** Tracks opportunity conversion trends across pipeline steps to flag operational friction points.
- **Marketing Attribution Analytics:** Charts deal volumes by their inbound channel source (e.g., Google Ads, Direct Form, Referral Traffic) to calculate marketing return-on-investment.
- **Cohort Conversion Velocities:** Breaks down deal closure rates month-by-month to analyze team pipeline stability over time.

V. ADVANCED ADMINISTRATIVE CUSTOMIZATION ENGINE

To avoid rigid column constraints and eliminate developer bottlenecks, the database layout incorporates a flexible customization engine. Company owners can tailor data schemas and workflow tracking to their unique business needs directly through administrative settings:

5.1 Dynamic Schema Custom Fields Engine

Admins can append custom variables onto core system profiles (Leads, Contacts, Deals) through an administrative control panel. The setup logic handles field configurations entirely through the UI without requiring code deployments or database migrations:

1. The Admin defines a new attribute, setting properties for `Field Label` (e.g., "GST Identification Code"), `Field Type` (e.g., `TEXT` , `NUMBER` , `DATE` , `BOOLEAN`), and the targeted module assignment (e.g., `LEADS`).
2. The system saves this rule under the `custom_field_definitions` metadata table.

3. When a rep accesses a lead card, the frontend queries these metadata schemas and updates profile forms dynamically.
4. User entries save directly as variable rows inside the `custom_field_values` metadata table, ensuring database stability.

5.2 Dynamic Custom Pipeline Stage Configurator

Admins can overwrite or re-arrange default pipeline tracking stages to map closer to their actual internal business logic. Through an administrative configuration panel, admins can add, rename, or re-order steps to match unique company processes:

- **Standard Service Structure Model:** `Lead` → `Prospect` → `Proposal Sent` → `Contract Executed`.
- **Heavy Outbound Enterprise Model:** `Inquiry Intake` → `Technical Demo Scheduled` → `Pricing Proposal Approved` → `Purchase Order Captured` → `Legal Contract Signed`.

The visual pipeline board queries these custom database rows to generate columns and structure drag-and-drop metrics automatically.

5.3 Dynamic Global Filtering & View Preservation System

To facilitate fast navigation across large datasets, filtering controls are standard across all application modules. Reps can filter listings by `Date Range`, `Account Owner`, `Conversion Status`, `Marketing Channel Source`, and any active `Custom Fields`. Users can save complex filter groups as a personal **Saved View Asset** (e.g., `"High Value Gujarat Enterprise Pipelines"` — filtering for values over ₹5 Lakhs with an active regional tag). Saved views appear directly in module sidebars for single-click access.

VI. SYSTEM NON-FUNCTIONAL REQUIREMENTS & GUARDRAILS

6.1 Performance and Target Latency Metrics

- **Page Load Response Times:** Core layout shell and dashboard widget elements must render within a 2-second envelope across modern data connections.
- **Search Engine Performance:** Typing strings inside the global search command bar must trigger lookups and return matching entries in under 500ms.
- **REST API Response Envelopes:** Standard database write and read operations across backend API routes must complete response processing in under 300ms.

6.2 System Security & Access Controls

- **Database Row-Level Security (RLS):** Strict row-level rules are configured directly on Supabase PostgreSQL tables. Every query validation check must verify the user's matching `organization_id` context before returning data blocks.
- **Enterprise Token Passports:** System API calls require secure JavaScript Web Token (JWT) authorization tokens, renewed continuously through secure session cookies.
- **Infrastructure Data Encryption:** Client records use AES-256 bit encryption standards for cold storage, while active network traffic uses TLS 1.3 transport encryption.

VII. STRATEGIC COMPREHENSIVE ENGINEERING ROADMAP (ALL PHASES)

To maintain development alignment across subsequent engineering cycles, the complete platform evolution roadmap is structured across four sequential phases:

- Phase 1: Foundation Baseline (Current Specification — Months 1-2):** Core multi-tenant architecture setup, relational data schemas, visual pipelines, tasks, basic analytics dashboards, dynamic custom attributes, global search tools, and saved database views.
- Phase 2: Omni-channel Automation Gateways (Months 3-4):** Integration of Meta’s WhatsApp Business API, background IMAP email sync tracking, business call logging engines, custom pricing quotation builders, and automated messaging drip sequences.
- Phase 3: Contextual AI Intelligence Layer (Months 5-6):** Introduction of predictive lead scoring, automated speech-to-text call transcription, contact record auto-enrichment scripts, and automated proposal engagement tracking.
- Phase 4: Autonomous Sales Operating Optimization (Months 7+):** Integration of the **AI Next Best Action Command Engine**, live call objection flash assistance systems, autonomous WhatsApp conversational agents, and predictive cash flow reporting.

--- END OF PRODUCTION TECHNICAL FUNCTIONAL REQUIREMENTS DOCUMENT ---

This blueprint represents the formal operational master copy for engineering deployment.
Authorized for production database indexing, application service provisioning, and core repository initiation.